- Compute Match — Reservation Request Lifecycle and Status Reference
  - Two levels of status
  - Reservation request statuses
  - Offering statuses (per NCP)
  - Catalog / multi-NCP flow, step by step
  - Custom request flow
  - Capacity rules that affect requests
  - Configuration defaults
  - Allocation statuses

Compute Match — Reservation Request Lifecycle and Status Reference

This document explains, end to end, how a reservation request moves through statuses, who triggers each change, and which timers apply.

Two levels of status

1. **Reservation request status** — the status of the whole customer request.
2. **Offering status** — the status of each NCP's slice (`ReservationRequestOffering`). A request sent to 3 NCPs has 3 offerings.

Do not confuse these with **GPU offering status** (inventory listing status: `ACTIVE` , `PENDING_REVIEW` , `REJECTED`).

Reservation request statuses

Backend enum `ReservationRequestStatus` (`backend/core/reservation_request/reservation_request_status.py`):

Backend value	UI label	Meaning
`Open`	Open	Submitted; not yet routed (custom requests start and may stay here)
`AwaitingProviderAcceptance`	Awaiting NCP Acceptance	Routed to one or more NCPs, waiting for a response

Backend value	UI label	Meaning
`InProcess`	In Process	At least one NCP accepted; negotiation under way
`Contracted`	Contracted	An NCP confirmed the contract (legacy `Approved` also shows as Contracted)
`Rejected`	Rejected	All NCPs rejected
`Expired`	Expired	Deadlines passed without action
`Cancelled`	Cancelled	Cancelled by the customer
`InReview`	In Review	Custom request being worked on by admins
`Matched`	Matched	Custom request matched to an NCP by admins
`Unserviceable`	Unserviceable	Custom request closed as not serviceable

The UI also shows **Awaiting Provider Approval** and **Awaiting Customer Approval** for older proposal/counteroffer flows.

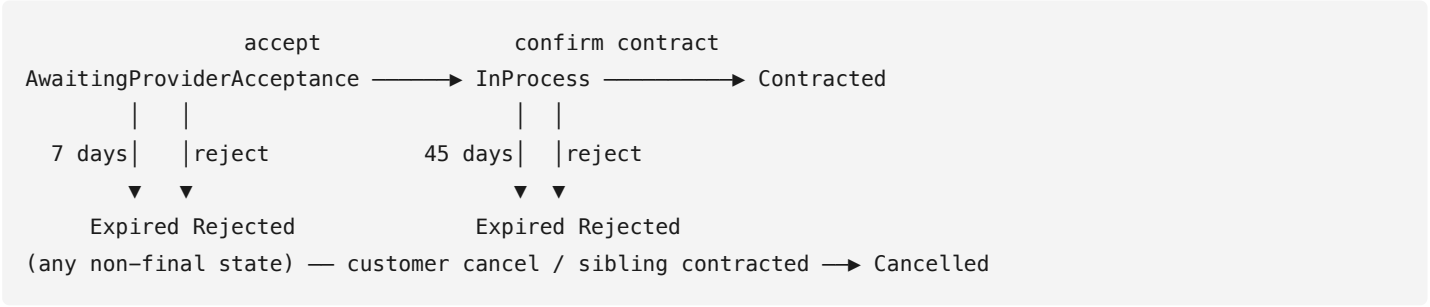
Offering statuses (per NCP)

`OfferingStatus`: `AwaitingProviderAcceptance`, `InProcess`, `Contracted`, `Expired`, `Rejected`, `Cancelled`.

Catalog / multi-NCP flow, step by step

1. **Customer submits** (`POST /api/v1/reservation-request`). The backend creates the request and the customer's first proposal, then routes it: one offering per selected cluster (backend limit 10, UI limit 3 by default).
2. Each offering starts as **AwaitingProviderAcceptance** with `acceptance_expiry_date = now + 7 days`. The request becomes **AwaitingProviderAcceptance**.
3. Temporal starts a parent `ReservationRequestWorkflow` and one child `OfferingLifecycleWorkflow` per offering. The customer gets a "request received" email; each NCP gets a magic-link email.
4. **NCP accepts** → offering **InProcess**, `contract_expiry_date = now + 45 days`, NCP and customer receive an introduction email. The first acceptance moves the request to **InProcess**.
5. **NCP counteroffers** → a new proposal is recorded (price-only changes update in place; material changes create a new proposal in history).
6. **NCP confirms contract** → offering **Contracted**, an **Allocation** is created (status `scheduled`), the request becomes **Contracted**, and sibling offerings are **Cancelled** with "superseded" emails.

7. **NCP rejects** (remarks required) → offering **Rejected**. If all offerings are rejected, the request is **Rejected**.
8. **Timers**: if an NCP does not accept in 7 days → offering **Expired**. If no contract in 45 days → offering **Expired**. When every offering has expired, the customer gets one consolidated expiry email and the request becomes **Expired**. Reminders go out 3 days before each deadline.
9. **Customer cancels** → request **Cancelled**; NCPs are notified.
10. **Extensions**: a Platform Admin (scope `accept_contract_extension`) can extend acceptance or contract deadlines by 1–200 days; this can revive an Expired offering.



Custom request flow

A custom request has no selected cluster (`gpu_offering_id` is empty and there are no offerings).



- Only admins with `approver_manage_custom_reservation_request` move it forward.
- Only the customer cancels.
- Never returns to Open; Matched, Unserviceable and Cancelled are final.
- Custom requests never expire.
- Each transition records a timestamp and optional NCP name, NCP region and comment, and emails the customer.

Capacity rules that affect requests

- Capacity windows of the same type (AVAILABLE or FUTURE) cannot partially overlap; at most one open-ended window per type.
- Availability checks use the **upper bound** of the tenure band (for example `2–3` years is checked as 3 years) to be conservative.
- Allocations only subtract from the matching capacity type.
- An optional cap (`MAX_PENDING_REQUESTS_PER_REGIONAL_CAPACITY`, default 0 = unlimited) hides clusters with too many pending requests and rejects new requests against them.

- GPU count per request must be between 1 and 100,000.
- Contracted price must be zero or positive and is only visible to NCPs and NVIDIA admins.

Configuration defaults

From `backend/multi_provider_config.py` (environment prefix `MULTI_PROVIDER_`):

Setting	Default	Meaning
`ACCEPTANCE_EXPIRY_DAYS`	7	NCP acceptance window
`IN_PROCESS_EXPIRY_DAYS`	45	Contract negotiation window
`MAX_GPU_OFFERINGS_PER_REQUEST`	10	Backend cap on offerings per request (UI default is 3, `NEXT_PUBLIC_MAX_MULTI_NCP_SELECTION_COUNT`)
`MAX_PENDING_REQUESTS_PER_REGIONAL_CAPACITY`	0	0 means unlimited
`MAGIC_LINK_EXPIRY_SECONDS`	3600	Magic link valid for 1 hour
`MAGIC_LINK_REGENERATION_WINDOW_DAYS`	7	Resend allowed for 7 days after routing

Reminder lead time is `EXPIRY_REMINDER_LEAD_DAYS = 3` in the Temporal workflow.

Allocation statuses

`scheduled`, `provisioning`, `running`, `stopping`, `complete`.